

SaaS de Gestión de Créditos — Arquitectura y Finanzas

Documento técnico de arquitectura, operación y modelo financiero

Versión 1.2 · 15 de septiembre de 2026 · Documento de trabajo interno

BASE DE REQUERIMIENTOS DEL PROYECTO

SAAS DE GESTIÓN DE CRÉDITOS Y COBRO DIARIO

Documento Técnico de Arquitectura, Operación y Modelo Financiero

Versión 1.2 — Documento base de definición (nombres en español + calidad)

Preparado para: uso interno, planificación del producto y equipo de desarrollo

Fecha: 15 de septiembre de 2026

DOCUMENTO DE TRABAJO. Este documento formaliza la definición inicial del producto. Las decisiones marcadas como *por definir* deben confirmarse antes de convertirlas en compromisos de implementación.

Cambios respecto a v1.0: frontend Angular (en lugar de Next.js/React); monorepo dockerizado desde el día uno; Swagger UI y colecciones Postman en el flujo de desarrollo; árbol de directorios corregido; esquema de BD con entidades, campos, relaciones, índices y transacciones. v1.2: tablas y código de dominio en español; tests como criterio de entrega.

Tabla de contenido

1. Objetivo y principios de decisión tecnológica
2. Arquitectura propuesta
3. Estructura del proyecto
4. Stack tecnológico por capa
5. Base de datos y persistencia
6. Autenticación, autorización y multiempresa
7. Pagos, suscripciones y facturación
8. Seguridad técnica
9. Infraestructura y despliegue
10. Observabilidad, backups y continuidad
11. Estimación inicial de infraestructura
12. Modelo comercial y unit economics
13. Planes y propuesta comercial
14. Proyección de escenarios
15. Roadmap técnico
16. Riesgos técnicos y de negocio
17. Decisiones pendientes

1. Objetivo y principios de decisión tecnológica

Este documento complementa el SRS: define cómo construir, operar y monetizar el producto.

- Priorizar web responsive/PWA (Angular) antes que aplicación móvil nativa.
- Minimizar infraestructura administrada manualmente.
- Usar servicios administrados para base de datos, hosting y observabilidad.
- Mantener costos bajos antes de la tracción.
- Arquitectura modular, no microservicios innecesarios en el MVP.
- Multiempresa desde el inicio.
- Separar procesos pesados cuando sea necesario.
- Medir costos por cliente para conocer el margen real de cada plan.
- Trabajar desde el inicio con un **monorepo dockerizado** que contenga backend y frontend, para garantizar un entorno de desarrollo reproducible entre integrantes del equipo.
- Documentar y probar los endpoints desde el inicio (Swagger UI/OpenAPI + colecciones Postman), antes de considerar un módulo como terminado.

La prioridad tecnológica es que el cobrador tenga una experiencia rápida y confiable desde celular, mientras el administrador dispone de un panel completo desde computador.

2. Arquitectura propuesta

NAVEGADOR WEB/PWA → FRONTEND ANGULAR → API/BACKEND → POSTGRESQL; procesos programados/asíncronos → WORKERS → NOTIFICACIONES/REPORTES; servicios externos → PAGOS/FACTURACIÓN.

Componente	Responsabilidad
Frontend (Angular)	Interfaz responsive de administrador y cobrador (SPA/PWA).
Backend/API	Reglas de negocio, autorización, clientes, créditos, cobros y suscripciones. Expone OpenAPI/Swagger UI.
Base de datos	Persistencia transaccional PostgreSQL.
Workers	Reportes, notificaciones, tareas programadas y futuras conciliaciones.
Storage	Comprobantes/archivos si se incorporan (object storage, no BYTEA).
Pagos	Checkout y eventos de suscripción.
Observabilidad	Errores, logs y métricas.
Pruebas de API	Colecciones Postman por módulo y ambiente.

Decisión: no se recomienda Kubernetes ni microservicios para el MVP. La modularidad del monorepo permitirá separar componentes cuando exista una razón real.

3. Estructura del proyecto

El proyecto se organiza como un monorepo dockerizado desde el inicio: backend y frontend viven en el mismo repositorio, cada uno con su propio Dockerfile, y se levantan juntos en local mediante Docker Compose. Esto evita desalineación de versiones entre capas y facilita el onboarding de nuevos desarrolladores.

```
creditos-saas/  
apps/
```

web/ src/ Dockerfile	# Frontend responsive/PWA (Angular)
api/ src/ docs/swagger/ Dockerfile	# Backend / API # OpenAPI servida como Swagger UI
workers/ src/ Dockerfile	# Jobs asincronos
packages/ db/ shared-types/ ui/ config/ tests/ postman/ infra/ docker-compose.yml package.json	# Esquema, migraciones y acceso a datos # Tipos compartidos entre apps # Componentes reutilizables (Angular) # Configuración y validaciones compartidas # Colecciones Postman (por módulo y ambiente) # Orquestación local: web + api + workers + db

La estructura facilita compartir tipos y mantener coordinados frontend, backend y paquetes comunes, y permite que cualquier desarrollador levante el entorno completo con un solo comando de Docker Compose.

Ubicación de la fuente de verdad del modelo: `docs/00-esquema-bd.md` (y su réplica en `packages/db` cuando exista el código).

4. Stack tecnológico por capa

Capa	Propuesta	Motivo
Frontend	Angular + TypeScript + Angular Material y/o Tailwind CSS	Framework robusto orientado a SPA responsive/PWA, con estructura modular (rutas, servicios, guards) afín a un dominio con varios roles (administrador/cobrador).
Backend	Node.js + TypeScript + NestJS (alternativa: Fastify)	API modular, mismo lenguaje que el front, OpenAPI de primer nivel (@nestjs/swagger).
DB	PostgreSQL administrado	Modelo relacional para créditos/cuotas/pagos y transacciones.
Acceso a datos	Prisma (recomendado) en packages/db	Migraciones versionadas y tipos compartibles.
Auth	NestJS Auth + JWT, o Supabase Auth equivalente	Reducir superficie de autenticación sin perder control de tenant/rol.
Contenerización	Docker + Docker Compose	Monorepo dockerizado (backend y frontend) desde el inicio; entorno reproducible y base de CI/CD.
Documentación de API	Swagger UI / OpenAPI	Documentación viva y navegable de cada endpoint, generada junto con el backend.
Pruebas de endpoints	Colecciones Postman	Verificación funcional durante el desarrollo, por módulo y ambiente (local/staging).
Tests automatizados	Vitest (API) + specs Angular; <code>npm test</code> en raíz	Puerta de entrega: sin verde el cambio no está hecho.

Capa	Propuesta	Motivo
Frontend hosting	Vercel, Netlify o equivalente (build Angular)	Despliegue sencillo.
Backend hosting	Railway / Render / Fly.io o equivalente	Administrado y escalable.
Jobs	Workers propios + cron, o Trigger.dev/Inngest	Tareas programadas.
Errores	Sentry o equivalente	Monitoreo.
Repositorio	GitHub	Control de versiones.

La elección definitiva de hosting puede cambiar por costos, experiencia o requisitos de producción, manteniendo las responsabilidades arquitectónicas. **Angular, PostgreSQL, Docker Compose, Swagger UI y Postman son decisiones de esta versión.**

5. Base de datos y persistencia

PostgreSQL es la opción recomendada para el núcleo transaccional porque clientes, créditos, cuotas y pagos tienen relaciones estructuradas y requieren consistencia.

Convenciones: PK UUID, timestamps TIMESTAMPTZ, montos NUMERIC(18,2), tasas NUMERIC(8,4), nombres en **español snake_case**. Toda tabla de negocio lleva `negocio_id` (salvo catálogos globales y usuarios de plataforma).

5.1 Diagrama relacional

```

negocios (1) ---- (N) usuarios
negocios (1) ---- (N) clientes
clientes (1) ---- (N) creditos
creditos (1) ---- (N) cuotas
cuotas (1) ----- (N) pagos
creditos (1) ---- (N) asignaciones -- (N):(1) usuarios [rol cobrador]
negocios (1) ---- (1) suscripciones -- (N):(1) planes
usuarios/negocios (1) - (N) auditorias

```

Reservadas V1: `notas_cobro` (observaciones de cobro), `comprobantes_pago` (comprobante en object storage).

5.2 Catálogos iniciales

Catálogo	Valores
<code>estado_negocio</code>	activo, suspendido, cancelado
<code>rol_usuario</code>	propietario, administrador, cobrador, soporte, admin_plataforma
<code>estado_usuario / estado_cliente</code>	activo, inactivo
<code>periodicidad_credito</code>	diaria, semanal, quincenal, mensual
<code>estado_credito</code>	activo, pagado, mora, anulado, refinanciado
<code>estado_cuota</code>	pendiente, pagada, parcial, mora, anulada
<code>estado_pago</code>	valido, anulado
<code>estado_asignacion</code>	activa, finalizada
<code>estado_plan</code>	activo, inactivo
<code>estado_suscripcion</code>	activa, pago_fallido, cancelada, suspendida
<code>tipo_documento (abierto)</code>	CC, CE, NIT, PASAPORTE, OTRO

Catálogo	Valores
metodo_pago (abierto)	efectivo, transferencia, nequi, daviplata, otro

5.3 Entidades y campos

negocios — cliente empresarial del SaaS. id UUID PK; nombre_comercial VARCHAR(180); razon_social VARCHAR(180) nullable; documento_fiscal VARCHAR(40) nullable; moneda CHAR(3) default COP; estado estado_negocio; configuracion JSONB default {}; fecha_creacion; fecha_actualizacion. Unique parcial de documento_fiscal cuando no sea nulo.

usuarios — identidad. id UUID PK; negocio_id UUID FK negocios nullable; nombre VARCHAR(160); correo VARCHAR(180) unique; hash_contrasena VARCHAR(255); rol rol_usuario; estado estado_usuario; telefono VARCHAR(30) nullable; fecha_creacion; ultimo_acceso nullable; fecha_actualizacion. CHECK: roles de negocio (propietario, administrador, cobrador) exigen negocio_id; roles de plataforma (soporte, admin_plataforma) lo prohíben.

clientes — deudor. id UUID PK; negocio_id FK; nombre_completo; tipo_documento; numero_documento; telefono; direccion nullable; referencia_ubicacion nullable; estado; fecha_creacion; fecha_actualizacion; creado_por FK usuarios. Unique (negocio_id, tipo_documento, numero_documento).

creditos — crédito. id UUID PK; negocio_id FK; cliente_id FK; monto_principal NUMERIC(18,2) > 0; tasa_interes NUMERIC(8,4) >= 0 (validación legal pendiente); periodicidad; numero_cuotas >= 1; fecha_desembolso DATE; estado; condiciones_originales JSONB **immutable** (RF-005); fecha_creacion; fecha_actualizacion; creado_por FK usuarios.

cuotas — cuota. id UUID PK; negocio_id FK; credito_id FK; numero_cuota; fecha_vencimiento DATE; monto_esperado; saldo_pendiente (>= 0 y <= monto_esperado); estado. Unique (credito_id, numero_cuota).

pagos — recaudo. id UUID PK; negocio_id FK; cuota_id FK; credito_id FK (redundante para consultas de cartera); cobrador_id FK usuarios; monto > 0; fecha_pago TIMESTAMPTZ; metodo_pago; estado; motivo_anulacion nullable; anulado_por nullable; fecha_anulacion nullable; fecha_creacion. Un pago valido no se edita: se anula y se registra uno nuevo. Si estado = anulado, motivo y anulado_por son obligatorios.

asignaciones — cartera-cobrador. id UUID PK; negocio_id FK; credito_id FK; cobrador_id FK usuarios; fecha_asignacion; fecha_fin nullable; estado; asignado_por FK usuarios. Unique parcial: una sola fila activa por credito_id. Reasignar cierra la activa y crea una nueva en la misma transacción.

planes — catálogo global. id UUID PK; codigo unique (emprendedor, profesional, empresarial); nombre; limite_cobradores (3 / 10 / 15); precio_implementacion; precio_mensual; características JSONB; estado; fecha_creacion.

suscripciones — servicio por negocio. id UUID PK; negocio_id unique FK; plan_id FK; estado; fecha_inicio; fecha Renovacion; fecha_cancelacion nullable; referencia_pago_externo nullable; timestamps.

auditorias — trazabilidad. id UUID PK; negocio_id nullable; usuario_id FK; entidad; entidad_id; accion; detalle JSONB; fecha; ip_origen nullable. Tabla append-only.

V1 notas_cobro: id, negocio_id, credito_id, cliente_id, cobrador_id, nota, fecha.

V1 comprobantes_pago: id, negocio_id, pago_id unique, clave_almacenamiento, url_archivo nullable, fecha_creacion. El archivo vive en object storage.

5.4 Índices iniciales

- negocio_id en todas las tablas multiempresa.
- clientes (negocio_id, tipo_documento, numero_documento) unique; búsqueda por nombre y teléfono.
- cuotas (negocio_id, fecha_vencimiento, estado) — cobros del día y mora.
- pagos (negocio_id, fecha_pago, estado) — recaudo diario.
- asignaciones (negocio_id, cobrador_id, estado) — mi cartera.
- creditos (negocio_id, cliente_id, estado).

5.5 Transacciones obligatorias

Registrar pago: insertar pagos + recalculas cuotas.saldo_pendiente/estado + recalculas creditos.estado si aplica + insertar auditorias. Una sola transacción. Rollback si falla cualquier paso.

Anular pago: marcar anulado con motivo + recalculas cuota/crédito + auditar. No DELETE.

Reasignar cartera: cerrar asignación activa + insertar nueva + auditar.

5.6 Archivos

Comprobantes, si se incorporan, viven en almacenamiento de objetos separado de PostgreSQL, referenciados desde comprobantes_pago.

El detalle canónico (tipos exactos, CHECKs y mapeo a RF/RC/RA) está en docs/00-esquema-bd.md.

6. Autenticación, autorización y multiempresa

- El usuario inicia sesión y recibe una sesión segura (JWT o sesión del proveedor).
- Cada solicitud protegida identifica usuario, negocio (negocio_id) y permisos.
- El backend no debe confiar únicamente en filtros del frontend.
- Las consultas protegidas deben aplicar negocio_id.
- Los cobradores solo acceden a su cartera (asignaciones.estado = activa).
- Los administradores acceden según rol.
- El panel del proveedor SaaS usa autorización independiente (negocio_id nulo).
- Se recomienda 2FA para roles administrativos de alto privilegio.
- Las sesiones deben poder expirar y revocarse.

7. Pagos, suscripciones y facturación

El producto se comercializa como suscripción recurrente; la implementación inicial es un cobro separado.

Evento	Resultado
Suscripción creada	Crear/activar registro en subscriptions.
Pago aprobado	Mantener servicio activo.
Renovación	Actualizar periodo.
Pago fallido	Marcar pago_fallido y notificar.
Cancelación	Registrar fecha_cancelacion.
Upgrade	Actualizar plan_id y capacidad.
Downgrade	Aplicar límites según política.

Para Colombia se debe definir con contador/asesor tributario el mecanismo de facturación electrónica y tratamiento fiscal aplicable.

8. Seguridad técnica

- HTTPS obligatorio.
- Secretos y claves fuera del código fuente.
- Variables de entorno separadas por ambiente (Compose, staging, producción).
- Cifrado de datos sensibles cuando corresponda.

- Control de acceso por rol y negocio (negocio_id).
- Rate limiting.
- Logs sin contraseñas, tokens ni información innecesaria.
- Backups cifrados.
- Dependencias actualizadas.
- Auditoría de operaciones financieras críticas.
- Principio de mínimo privilegio para soporte.
- Ejemplos de Swagger y Postman con placeholders (user@example.com, <password>).

9. Infraestructura y despliegue

Para la primera etapa se recomienda infraestructura administrada y de bajo costo. El volumen descrito no exige un servidor dedicado grande; la prioridad es confiabilidad, backups y capacidad de crecimiento.

Servicio	Función	Criterio
Docker + Docker Compose	Entorno de desarrollo local	Monorepo dockerizado (web + api + workers + db); sin costo, estándar desde el MVP-1
Vercel / equivalente	Frontend Angular	Free tier / plan inicial según tráfico
Railway / Render / equivalente	API + workers	Escalar según uso
PostgreSQL administrado	DB	Free tier / plan inicial y luego escalar
Storage	Comprobantes	Pago por almacenamiento
Sentry / equivalente	Errores	Free tier inicial
Dominio	Acceso	Costo anual
Backups	Continuidad	Incluidos o almacenamiento separado

10. Observabilidad, backups y continuidad

- Monitorear disponibilidad de frontend y API.
- Registrar errores con identificador de correlación.
- Monitorear pagos y webhooks.
- Monitorear trabajos programados.
- Definir frecuencia de backup.
- Probar restauración, no solo creación.
- Definir recuperación ante caída.
- Definir quién recibe alertas.
- Conservar auditoría de cambios críticos.

11. Estimación inicial de infraestructura

Presupuesto de planificación inicial: \$150.000–\$500.000 COP/mes para infraestructura y servicios técnicos básicos, antes de costos adicionales variables. No es una cotización definitiva.

Concepto	Rango mensual
Hosting/API/servicios base	\$100.000 – \$250.000 COP
Backups/almacenamiento	\$30.000 – \$80.000 COP
Dominio equivalente mensual	~\$10.000 COP
Monitoreo/herramientas	\$0 – \$100.000 COP
Total orientativo	\$150.000 – \$500.000 COP

El costo real debe medirse por consumo y aumentar de acuerdo con el volumen.

12. Modelo comercial y unit economics

Modelo: implementación inicial + suscripción mensual. No se recomienda vender definitivamente el código como modelo principal.

Plan	Implementación	Mensualidad	Recurrente anual por cliente*
Emprendedor	\$1.250.000	\$250.000	\$3.000.000
Profesional	\$2.500.000	\$425.000	\$5.100.000
Empresarial	\$3.500.000	\$750.000	\$9.000.000

*Sin cancelaciones, descuentos, impuestos, comisiones de pago ni costos variables.

La implementación ayuda a recuperar configuración; la mensualidad sostiene infraestructura, soporte, mantenimiento y evolución.

13. Planes y propuesta comercial

Emprendedor — \$250.000/mes: hasta 3 cobradores; clientes, créditos, cuotas, cobros diarios, historial, cartera, dashboard básico, soporte y actualizaciones.

Profesional — \$425.000/mes: hasta 10 cobradores; funciones anteriores + reportes avanzados, control por cobrador, exportaciones y soporte prioritario.

Empresarial — \$750.000/mes: hasta 15 cobradores; administración avanzada, múltiples administradores, indicadores, configuración personalizada y soporte prioritario.

Implementación: \$1.250.000 / \$2.500.000 / \$3.500.000 respectivamente.

"No tienes que comprar ni mantener un sistema. Contratas una plataforma para administrar créditos y cobros diarios, con acceso desde celular y computador, soporte y evolución continua."

La propuesta debe vender eficiencia, control y trazabilidad; no prometer rentabilidad de la actividad crediticia.

14. Proyección de escenarios

Ejemplos simplificados de ingresos recurrentes brutos, antes de impuestos, soporte, comisiones, infraestructura y cancelaciones. Hipótesis: clientes en plan Profesional.

Escenario	Clientes	Mensual recurrente	Anual recurrente
Inicial	5	\$2.125.000	\$25.500.000
Tracción	10	\$4.250.000	\$51.000.000

Escenario	Clientes	Mensual recurrente	Anual recurrente
Crecimiento	20	\$8.500.000	\$102.000.000
Escala	50	\$21.250.000	\$255.000.000

Implementaciones Profesionales: 5 clientes = \$12.500.000; 10 = \$25.000.000; 20 = \$50.000.000, suponiendo pago completo.

El modelo financiero definitivo debe incorporar churn, descuentos, impuestos, comisiones, soporte, marketing, salarios e infraestructura.

15. Roadmap técnico

Fase	Trabajo
MVP-1	Monorepo dockerizado (Docker Compose), frontend Angular responsive, backend con Swagger UI, colección Postman inicial, PostgreSQL, autenticación, negocio y harness de tests.
MVP-2	Clientes, cobradores, créditos, cuotas, asignación y reglas.
MVP-3	Cobro diario, actualización transaccional, cartera, dashboard y auditoría.
MVP-4	Planes, suscripciones, checkout/webhooks y administración SaaS.
Piloto	Backups, monitoreo, hardening, pruebas y operación real.
V1	Reportes, exportaciones, comprobantes, notificaciones y UX móvil refinada.
V2+	Pagos integrados, conciliación, automatizaciones e integraciones.

16. Riesgos técnicos y de negocio

Riesgo	Impacto	Mitigación
Reglas mal definidas	Alto	Cerrar reglas antes del desarrollo.
Errores en pagos	Muy alto	Transacciones, validaciones, auditoría, Swagger/Postman y pruebas.
Fuga entre negocios	Crítico	Aislamiento por negocio_id y pruebas automatizadas de seguridad.
Caída durante cobro	Alto	Infraestructura administrada, monitoreo y backups.
Entorno irreproducible	Alto	Monorepo dockerizado obligatorio desde MVP-1.
Soporte crece demasiado	Medio/alto	Planes con límites y soporte diferenciado.
Precios insuficientes	Alto	Medir unit economics.
Cambios legales	Alto	Validación jurídica/contable.
Scope creep	Alto	MVP estricto y roadmap.

17. Decisiones pendientes

- Nombre y marca comercial.
- Campos definitivos de clientes.
- Identificación y validaciones.
- Reglas exactas de cálculo y tasas.
- Periodicidades.
- Redondeos.
- Mora y recargos.
- Pagos parciales/anticipados.
- Refinanciación/reestructuración.
- Política comercial de anulación/corrección de pagos.
- Métodos de pago y comprobantes.
- Modo offline/PWA.
- Notificaciones.
- Pasarela de suscripción.
- Proveedor de facturación electrónica.
- Política de cancelación/suspensión.
- SLA y soporte.
- Requisitos legales y tributarios.
- Presupuesto máximo mensual.
- Objetivo de clientes a 12 meses.
- Confirmación de NestJS vs Fastify (recomendación actual: NestJS por Swagger).
- Confirmación de Prisma vs otro ORM.
- Proveedor de autenticación (JWT propio vs Supabase Auth).